Episodic Memory for Language Agents: A Minimal Architecture

M. Okafor, L. Brandt, S. Ishikawa · Workshop on Long-Horizon Agents · 2025 (demo document)

Abstract. Language agents built on large language models forget everything between calls: the context window is the only state they carry. We describe a minimal architecture that gives an agent an episodic memory, a store of time-stamped events written after each step and retrieved before each decision. The architecture has three components: a write path that summarises and scores each event, a read path that ranks stored events by relevance, recency and importance, and a consolidation process that periodically distils recurring episodes into semantic facts. On a suite of long-horizon tasks the agent with episodic memory completes 71% of tasks versus 38% for a context-window-only baseline, while using 40% fewer prompt tokens per step. We discuss failure modes, in particular retrieval of stale or misleading memories, and argue that forgetting is a feature rather than a bug.

1 Introduction

A language agent is a loop: observe, think, act. Each iteration is a fresh call to a model that has no memory of previous calls, so everything the agent needs to know must be placed in the prompt. For short tasks this is fine. For tasks that span hours, days or many sessions, the prompt fills up, the oldest observations fall off the edge, and the agent starts repeating work it has already done or contradicting decisions it has already made.

The obvious fix, a longer context window, does not solve the problem. Long windows are expensive, attention over very long inputs is unreliable, and a transcript is a poor representation of what happened: most of it is noise. What an agent needs is closer to what people have, a memory that keeps a small number of important things and lets the rest fade.

We take the vocabulary of cognitive psychology as a design guide. Tulving (1972) distinguished episodic memory, the memory of specific events situated in time, from semantic memory, general knowledge detached from the moment it was learned. Working memory (Baddeley and Hitch, 1974) is the small, active buffer used during reasoning. Procedural memory holds skills. We map each of these onto a component of an agent and show that the mapping yields a simple, effective architecture.

Our contributions are: (i) a minimal write/read/consolidate architecture for episodic memory; (ii) a retrieval score that combines relevance, recency and importance with a single tunable weight each; (iii) an evaluation on long-horizon tasks showing large gains over a context-window baseline; and (iv) an analysis of the characteristic ways in which memory-equipped agents fail.

2 Four kinds of memory

Table 1 summarises the four kinds of memory we distinguish, their counterpart in an agent, and the component that implements each one. The distinction that matters most in practice is between working memory, which is the context window and is therefore free but small, and the three long-term stores, which are external, effectively unbounded, and only useful through retrieval.

Kind	What it holds	In an agent	Lifetime
Working memory	The current task, recent observations, the plan	The context window	One call
Episodic memory	Specific events with a time stamp: what happened, when, with whom	A store of summarised events, retrieved by similarity and recency	Days to months; decays

Semantic memory	Facts and preferences abstracted from experience	A profile or knowledge base, updated by consolidation	Until contradicted
Procedural memory	How to do things	Tools, prompts, playbooks; occasionally learned skills	Stable

Table 1. Four kinds of memory and their counterparts in a language agent.

Episodic memory is the one most often missing. Working memory comes for free, semantic memory is usually hand-written into a system prompt, and procedural memory is the tool set. But nothing in a standard agent remembers that yesterday the user asked for the report in French, or that the deployment failed twice last week for the same reason. Those are episodes, and they are precisely what makes an assistant feel like it knows you.

3 Architecture

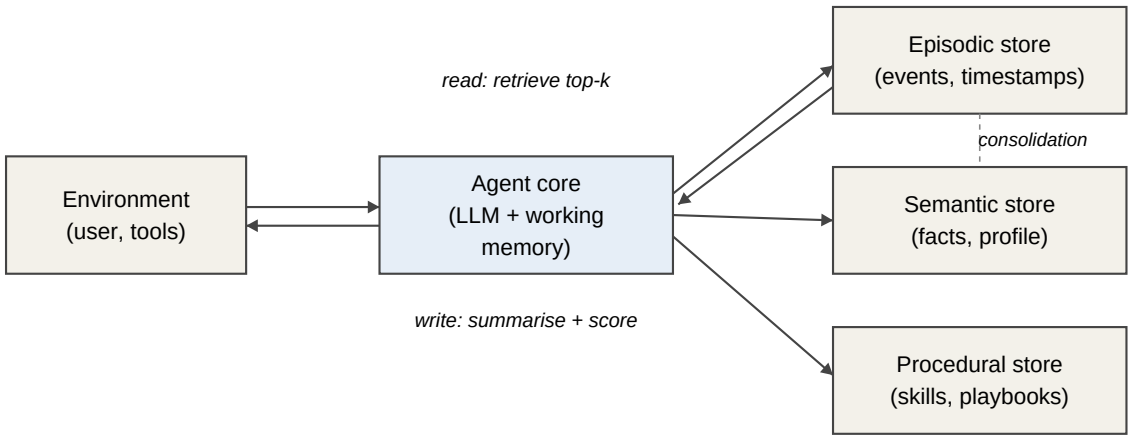

Figure 1. The agent core reads from and writes to three long-term stores. Consolidation moves recurring episodes into semantic facts.

3.1 Write path

After every step the agent emits an event: the observation it received, the action it took and the outcome. Raw events are too long and too noisy to store as they are. The write path passes each event through a summariser that produces a one- or two-sentence description, attaches a time stamp and an embedding, and asks the model for an importance score between 1 and 10. Importance is a judgement of how much the event would matter to a future decision: a user stating a hard constraint scores high, an intermediate tool call that succeeded uneventfully scores low. Events below a threshold are still stored, but their low importance makes them unlikely to be retrieved.

3.2 Read path

Before each decision the agent forms a query from its current goal and the last observation, and retrieves the top-k events by a combined score:

$$score(e) = \alpha \cdot relevance(e, q) + \beta \cdot recency(e) + \gamma \cdot importance(e)$$

where relevance is cosine similarity between the query and the event embedding, recency is an exponential decay of the event age with a half-life of one day, and importance is the stored score normalised to [0, 1]. We use $\alpha = 1.0$, $\beta = 0.5$ and $\gamma = 0.5$ throughout; the results are not sensitive to these weights within a factor of two. The retrieved events are rendered as a short bulleted list and placed in the prompt under a heading

"Relevant memories", ahead of the current observation. We retrieve k = 8 events by default.

3.3 Consolidation

Episodes accumulate. Left alone, the store grows without bound and the same fact is represented by dozens of near-duplicate events. Consolidation runs whenever the sum of importance scores since the last run exceeds a threshold. It clusters recent episodes, asks the model what general statements the cluster supports, and writes those statements to the semantic store. For example, five episodes in which the user corrected the agent for using metric units are consolidated into the fact "the user wants imperial units". The episodes are kept but their importance is halved, so that the fact rather than the events is what gets retrieved. This is the mechanism by which the agent learns preferences without anyone writing them down.

4 Evaluation

We evaluate on 120 long-horizon tasks in three domains: multi-session research assistance, software maintenance over a week of simulated commits, and personal scheduling with changing constraints. Each task requires information from an earlier session to be used in a later one. We compare four agents: a context-window-only baseline that keeps as much transcript as fits; a baseline that keeps a rolling summary of the transcript; our architecture without consolidation; and the full architecture.

Agent	Tasks completed	Prompt tokens per step	Contradictions per task
Context window only	38%	6,900	2.4
Rolling summary	52%	2,100	1.7
Episodic memory, no consolidation	66%	2,600	0.9
Full architecture	71%	2,400	0.6

Table 2. Results on 120 long-horizon tasks. Figures are illustrative; see the demo note in Section 7.

Episodic memory nearly doubles the completion rate of the context-window baseline while cutting prompt size by about 60%. The rolling summary is cheap but lossy: it tends to keep the most recent material and drop hard constraints stated early on. Consolidation adds five points, almost entirely from tasks where a preference stated in one session had to be honoured in another. A contradiction is a case where the agent takes an action that violates a constraint it had previously acknowledged; memory cuts these by three quarters.

5 Failure modes

Memory introduces failures that a memoryless agent cannot have. We observed four recurring kinds.

- **Stale memory.** The agent retrieves an event that was true and is no longer: an address that has changed, a decision that was reversed. Recency weighting helps but does not eliminate this; the fix is to write the reversal as a high-importance event and, at consolidation, let it overwrite the fact.
- **Retrieval miss.** The right memory exists but the query does not match it. This is most common when the user refers to something obliquely ("the thing from Tuesday"). Adding the time stamp to the embedded text recovers most of these cases.
- **Memory poisoning.** Content from a tool result or a web page is written to memory as if the user had said it, and later retrieved as a trusted preference. Events must record their provenance and untrusted sources should never be consolidated into facts.
- **Over-retrieval.** With a large store, the top-k list is filled with marginally relevant events that crowd out the current observation. Lowering k or raising the importance weight is a blunt fix; a better one is to let

the model request more memory rather than always providing it.

Forgetting, in this light, is a feature. An agent that remembers everything is an agent that retrieves the wrong thing. The decay in the recency term and the halving of importance at consolidation are both ways of forgetting gracefully.

6 Related work

The taxonomy of memory we use is standard in cognitive psychology (Atkinson and Shiffrin, 1968; Tulving, 1972; Baddeley and Hitch, 1974). Retrieval-augmented generation retrieves documents rather than the agent's own past, but the machinery is the same and our read path is a special case. Several recent agent frameworks include a memory module; what distinguishes ours is the insistence on a minimal design with an explicit consolidation step and an explicit account of failure modes.

7 Note on this document

This is a demonstration document written for the haoku knowledge workspace. The authors are fictional and the numbers in Table 2 are illustrative rather than measured. The architecture and the failure modes, however, describe real and common patterns in agent design and are intended to be read, cited and questioned as such.

References

Atkinson, R. C. and Shiffrin, R. M. (1968). Human memory: a proposed system and its control processes. In *The Psychology of Learning and Motivation*, vol. 2.

Baddeley, A. D. and Hitch, G. (1974). Working memory. In *The Psychology of Learning and Motivation*, vol. 8.

Tulving, E. (1972). Episodic and semantic memory. In *Organization of Memory*. Academic Press.